

UNIDAD N°3: ADMINISTRACIÓN DE PROCESOS (2da PARTE)

BLOQUE DE CONTROL DE PROCESOS (BCP)

El BCP es una estructura de datos creada por el SO, que se encarga de almacenar la información acerca de determinados elementos presentes durante la ejecución de un proceso (identificador, estado, prioridad, contador de programa, punteros a memoria, información de estado de E/S e información de auditoría). El BCP es una herramienta clave que permite al SO dar soporte a múltiples procesos y proporciona la multiprogramación.

Cuando un proceso se interrumpe, los valores del contador de programa u los registros del procesador se guardan en determinados campos del BCP y el estado de proceso se cambia a otro valor (bloqueado o listo). Hecho esto, el SO es libre de poner otro proceso en estado de ejecución. El contador de programa y los datos de contexto se recuperan y cargan en los registros del procesador y este proceso comienza a correr.

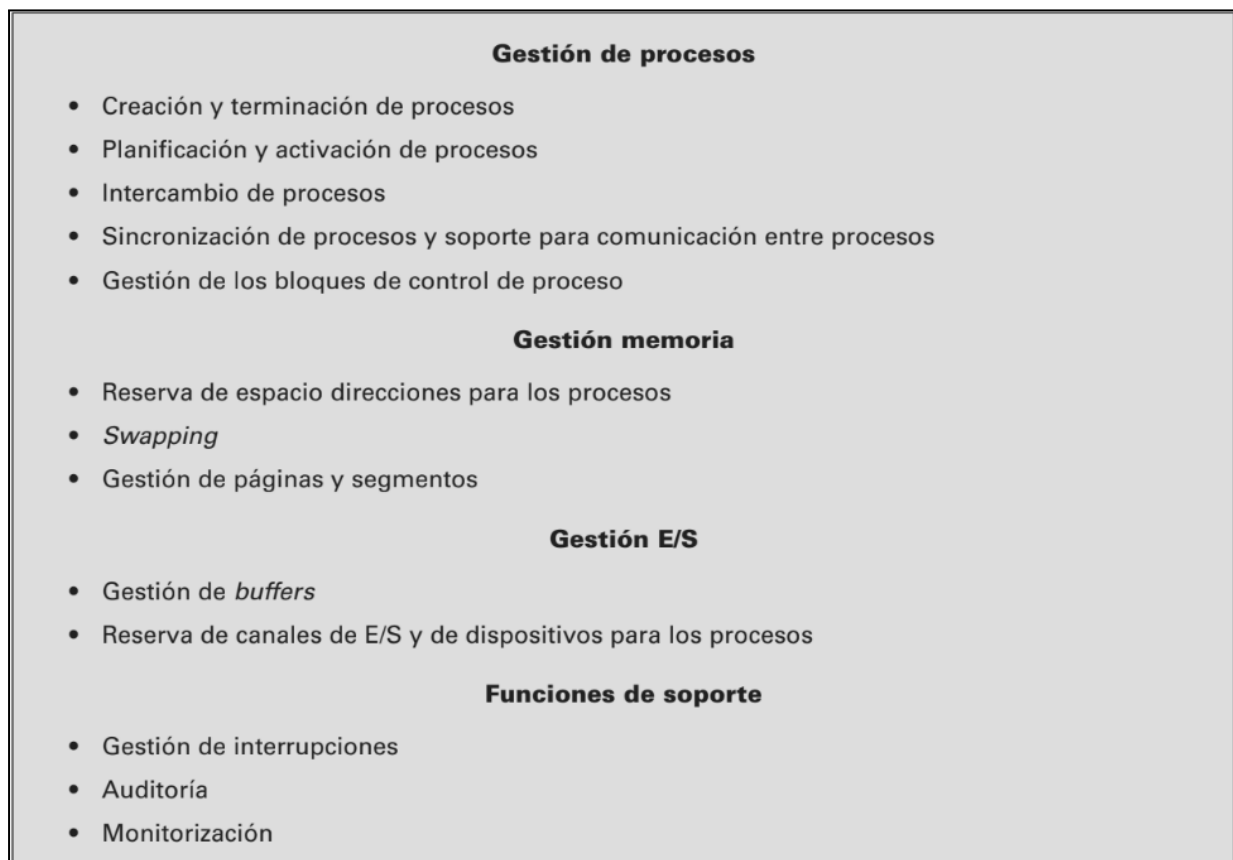
Como se anticipó, los elementos básicos que contiene el BCP son:

- **IDENTIFICADORES DE PROCESO:** es un número único que distingue a cada proceso. Tenemos tanto identificadores del proceso, del proceso padre que creó a este proceso y del usuario.
- **INFORMACIÓN DE ESTADO DEL PROCESADOR:** indica los contenidos de los registros del procesador, entre los cuales se destacan:
 - *Registros visibles por el usuario:* aquellos a los que se puede hacer referencia mediante el lenguaje máquina que ejecuta el procesador cuando está en modo usuario.
 - *Registros de estado y control:* incluye al contador de programa (contiene la dirección de la siguiente instrucción a ejecutar), los códigos de condición (producto de operaciones lógico-aritméticas) y la información de estado (incluye los flags de interrupciones habilitadas/deshabilitadas y el modo de ejecución).
 - *Punteros de pila:* una o más pilas LIFO asociadas a cada proceso. Se encarga de almacenar los parámetros y direcciones de retorno de los procedimientos y llamadas al sistema. El puntero apunta a la parte más alta de la pila.
- **INFORMACIÓN DE CONTROL DE PROCESO:** usada por el SO para controlar y coordinar varios procesos activos, dentro de la cuál tenemos:
 - *Información de estado y planificación:* información que el SO necesita para analizar las funciones de planificación. Algunos de sus elementos son: estado de proceso, prioridad, información relativa a planificación y eventos.
 - *Estructuras de datos.*
 - *Comunicación entre procesos.*
 - *Privilegios entre procesos.*
 - *Gestión de memoria.*

- *Recursos propios o controlados por el proceso.*

FUNCIONES DEL NÚCLEO DEL SISTEMA

El núcleo del SO, es la parte del mismo que engloba las funciones más importantes del sistema. Si tenemos en cuenta los modos de ejecución, el modo núcleo (también llamado modo control o modo sistema), es el que tiene más privilegios, es decir que podrán leer y modificar registros de control, instrucciones de E/S primitivas e instrucciones relacionadas con la gestión de memoria. Estos últimos, son los que se definen como “funciones típicas del núcleo del sistema operativo”.



- El *swapping* o memoria de intercambio, implica mover todo o parte del proceso de memoria principal al disco. Se aplica cuando ninguno de los procesos en MP se encuentra en estado Listo, procediendo a hacer el intercambio de los procesos Bloqueados al disco en la cola de Suspendidos.

CONTROL DE PROCESOS

MODOS DE EJECUCIÓN

En general, los procesadores proporcionan dos modos de ejecución, los cuales poseen más o menos privilegios. Los máximos privilegios que se pueden asignar son: la lectura y modificación de los registros de control, instrucciones de E/S primitivas e

instrucciones relacionadas con la gestión de memoria. Los modos de ejecución más importantes son:

- **Modo USUARIO:** es el modo menos privilegiado, en el que se ejecutan los programas de usuario. Usando este modo, no se tiene acceso a las estructuras propias del SO por motivos de seguridad.
- **Modo NÚCLEO:** el software tiene control completo sobre el procesador, de sus instrucciones, registros y memoria. Este es el modo más privilegiado de ejecución.

Para que el procesador sepa cual es el modo de ejecución, se cuenta con el llamado **PSW (Program Status Word)**, que es un bit que se encuentra en la palabra de estado de programa, e indica el modo. Este bit cambia como respuesta a determinados eventos (es decir que puede pasar de un modo al otro, de acuerdo a lo que el proceso necesite del sistema).

CREACIÓN DE PROCESOS

Los pasos que se siguen para la creación de procesos son:

1. **Asignar un identificador de proceso único al proceso (PID):** se añade una nueva entrada a la tabla primaria de procesos, que contiene una entrada por proceso.
2. **Reservar un espacio de memoria para el proceso:** esto incluye también a la imagen del proceso. Para esto, el SO conoce cuánta memoria se requiere para el espacio de direcciones privado (programas y datos) y para la pila de usuario. Esta información la puede brindar el padre del proceso tras la solicitud de creación de proceso. Además, se debe reservar el espacio para el BCP.
3. **Inicialización del BCP:** su información se inicializa con la mayoría de sus entradas valiendo "0" (cero), excepto el contador de programa y los punteros de pila del sistema. La prioridad, por defecto, se establece como baja, a no ser que se haya fijado lo contrario en la solicitud. Inicialmente, el proceso no debe contener ningún recurso, a no ser que se especifique o haya sido heredado del proceso padre.
4. **Establecer los enlaces apropiados:** se debe enlazar el proceso en la cola de procesos listos para su ejecución.
5. **Creación o expansión de otras estructuras de datos:** por ejemplo, mantener un registro de procesos que se pueden usar posteriormente.

CAMBIO DE PROCESOS

Un cambio de proceso puede ocurrir en cualquier instante en el que el SO obtiene el control sobre los procesos actualmente en ejecución. Se pueden distinguir dos tipos de interrupciones:

- **Interrupciones ordinarias,** por causa de algún evento que es independiente al proceso actualmente en ejecución. Inicialmente, su control es transferido al manejador de interrupciones, para luego ser transferido a una rutina del SO que se encarga de cada interrupción en particular. Dentro de estas encontramos:

- *Interrupción de reloj*: producidas cuando el proceso en ejecución ha extendido o no la unidad máxima de tiempo de ejecución.
 - *Interrupción de E/S*: el SO determina que acción de E/S ocurrió, y si esta constituye un evento por el cual están esperando uno o más procesos, y los mueve al estado de Listos.
 - *Fallo de memoria*: el procesador se encuentra con una referencia a una dirección de memoria virtual, a una palabra que no se encuentra en memoria principal. El SO trae el bloque con la referencia de la memoria secundaria a principal, mediante una operación de E/S, pasando finalmente el proceso inexistente al estado de Bloqueado.
- **Traps**: están asociadas a una condición de error o excepción generada dentro del proceso que está ejecutando. Con estos, el SO conoce si una condición de error o excepción es irreversible.
 - **Llamada al sistema**: es procedente del programa en ejecución.

Mecanismo	Causa	Uso
Interrupción	Externa a la ejecución del proceso actualmente en ejecución.	Reacción ante un evento externo asíncrono.
Trap	Asociada a la ejecución de la instrucción actual.	Manejo de una condición de error o de excepción.
Llamada al sistema	Solicitud explícita.	Llamada a una función del sistema operativo.

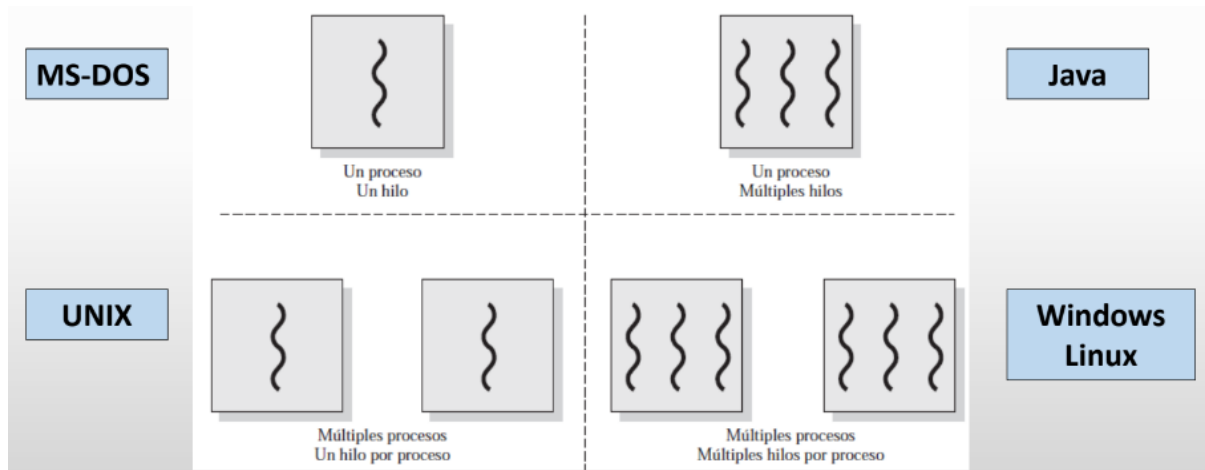
Además de estos cambios en los procesos, se pueden dar **cambios en el estado de los procesos**. Los pasos que se realizan para un cambio de proceso completo son:

1. Salvar el estado del procesador (contador de programa y otros registros).
2. Actualizar el BCP con el nuevo estado.
3. Mover el BCP a la cola correspondiente (Listo, Bloqueado, Listo/Suspendido, etc).
4. Seleccionar un nuevo proceso a ejecutar.
5. Actualizar el BCP elegido (estado Ejecutando).
6. Actualizar las estructuras de datos de gestión de memoria.
7. Restaurar el estado del procesador (registros del nuevo proceso).

HILOS (Thread, subproceso, proceso ligero)

Es el contexto de ejecución que se planifica de forma independiente, pero que comparte un mismo espacio de direcciones con otros hilos. Es una secuencia de tareas encadenadas que puede ser ejecutada por un SO, pudiendo hacerlo al mismo tiempo que otra tarea.

- Diferencia entre Hilos y Procesos: los procesos son, generalmente, independientes e interactúan a través de mecanismos de comunicación del sistema. Por otro lado, los hilos, generalmente comparten otros recursos de forma directa. Al compartir datos y espacios de direcciones, cambiar de un hilo a otro dentro del mismo proceso, se hace más rápidamente en relación a los procesos.

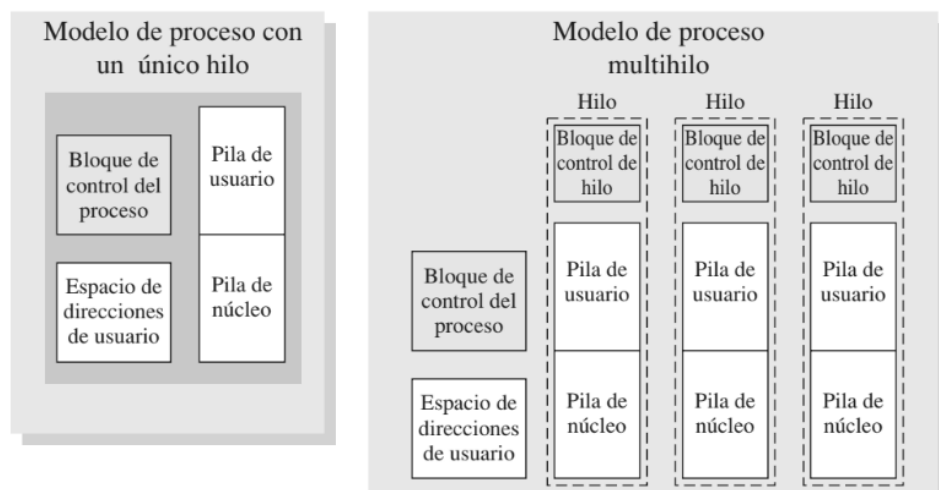


Los entornos **Monohilos**, son aquellos en donde existe un solo hilo de ejecución por proceso. Cada uno de los procesos, está representado por los elementos vistos inicialmente: BCP, pila de usuario, pila del núcleo y espacio de direcciones de usuario (los monohilos no se identifican con el concepto de hilo).

Por otro lado, los entornos **Multihilos**, son aquellos en los que el SO tiene la capacidad de mantener varios hilos de ejecución dentro de un mismo proceso. En estos, un proceso se define como la unidad de asignación de recursos y una unidad de protección, y se encuentra asociada a los siguientes elementos:

- Un espacio de direcciones virtuales que soporta la imagen del proceso.
- Acceso protegido a procesadores, procesos, archivos y recursos de E/S.
- Dentro de cada proceso, puede haber uno o más hilos, cada uno con un estado de ejecución, un contexto que se almacena cuando no está en ejecución, una pila de ejecución, un espacio de almacenamiento para variables locales y acceso a la memoria y recursos de su respectivo proceso.

En los entornos multihilos, existe un único BCP y un espacio de direcciones de usuario asociado al proceso, pero hay varias pilas separadas para cada hilo, así como un bloque de control para cada hilo con sus determinados valores de registros, prioridad y demás información de su estado. De esta forma, *todos los hilos de un proceso comparten el estado y los recursos de ese proceso, residen en el mismo espacio de direcciones y tienen acceso a los mismos datos.*



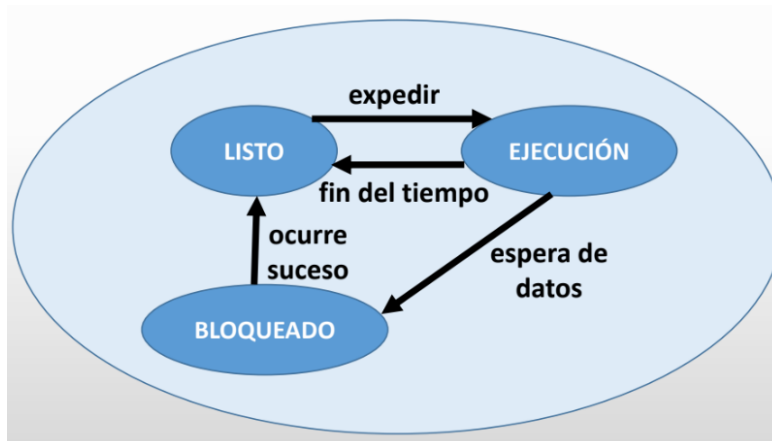
VENTAJAS DE LOS HILOS

- Facilita el diseño de aplicaciones que deben realizar distintas funciones simultáneamente.
- Creación de un nuevo hilo más rápido que un proceso.
- Terminación de un hilo más rápido que un proceso.
- Rapidez en cambiar de un hilo a otro.
- Los hilos de un mismo proceso se comunican sin acceder al núcleo.
- Es más eficiente relacionar un conjunto de hilos que un conjunto de procesos.

ESTADOS DE LOS HILOS

Las cuatro operaciones básicas relacionadas con los hilos que están asociadas con un cambio de hilo, son:

- **Creación:** al crear un proceso, se crea un hilo de dicho proceso. Luego, un hilo puede crear otro dentro del mismo proceso. Cada hilo creado, contará con los elementos propios de todos los hilos.
- **Bloqueo:** cuando un hilo necesita esperar por un evento se bloquea, almacenando todos los elementos de los hilos. El procesador puede pasar a ejecutar otro hilo en estado Listo, dentro del mismo proceso u otro diferente.
- **Desbloqueo:** cuando sucede el evento por el que el hilo está esperando, el hilo pasa a la cola de Listos.
- **Finalización:** cuando se completa un hilo, se liberan su registro de contexto y pilas.



HILOS DE NIVEL DE USUARIO Y DE NIVEL DE NÚCLEO

HILOS A NIVEL DE USUARIO (User-Level Thread, ULT)

En estos entornos, todo el trabajo de hilos lo realiza la aplicación en el espacio de usuario, además, la aplicación gestiona todo el trabajo de los hilos y el núcleo no es consciente de la existencia de los mismos. Para la gestión de ULT, se cuenta con un paquete de rutinas. La biblioteca de hilos, contiene código para crear y destruir hilos, para pasar

mensajes y datos entre ellos, para planificar la ejecución de los hilos, y para guardar y restaurar el contexto de los hilos.

Entre sus **ventajas** encontramos:

- El proceso no cambia a modo núcleo para realizar la gestión de hilos (se puede cambiar de hilo sin tener que cambiar a modo núcleo), ahorrando la sobrecarga de dos cambios de modo (usuario a núcleo; usuario a usuario).
- La planificación es realizada por una biblioteca de hilos, sin tener que recurrir al planificador del SO.
- Los ULT se pueden ejecutar en cualquier SO, sin necesidad de hacer cambios en el nuevo núcleo para darle soporte al ULT. Además, la biblioteca de hilos es compartida por todas las aplicaciones.

Entre sus **desventajas** encontramos:

- Si se bloquea a un hilo, se bloquean todos los hilos del proceso (muchas de las llamadas al sistema son bloqueantes).
- No se pueden aprovechar las ventajas del multiprocesamiento, debido a que el núcleo asigna el proceso a un solo procesador al mismo tiempo, pudiendo ejecutar un hilo del proceso en un determinado momento.

HILOS A NIVEL DE NÚCLEO (Kernel-Level Thread, KLT)

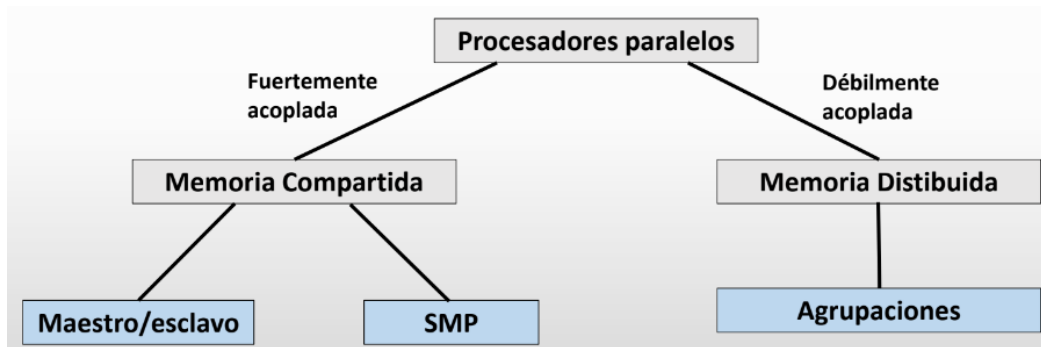
En este entorno, el núcleo es quien gestiona todo el trabajo de gestión de hilos. En estos, la planificación se realiza en función de los hilos. Esto último, trae consigo las **ventajas** de este enfoque (que resuelven los inconvenientes del enfoque ULT):

- El núcleo puede planificar simultáneamente varios hilos de un mismo proceso en múltiples procesadores.
- Si se bloquea un hilo de un proceso, se puede planificar otro hilo del mismo proceso.
- Las funciones del núcleo del SO se pueden implementar con multihilos.

Por otro lado, su principal **desventajas** es que requiere un cambio a modo núcleo para pasar de un hilo a otro.

SMP - MICRONÚCLEOS

Los procesadores, vistos a nivel de micro-operaciones, generan múltiples señales de control al mismo tiempo al ejecutar un programa. Esto permite que se desarrolle el llamado “Procesamiento en paralelo”, el cual se da en casos como la búsqueda y ejecución de operaciones. Uno de los enfoques que más aprovechan este paralelismo, es el **Multiprocesamiento simétrico (SMP)**.



Dicho de otro modo, el paralelismo puede verse como un conjunto de procesadores que ejecutan simultáneamente diferentes secuencias de instrucciones en diferentes conjuntos de datos. La forma de comunicación entre los procesadores se puede subdividir, gracias a la incorporación de memorias dedicadas en cada uno. Pero si además estos procesadores tienen una memoria compartida, y los procesadores se comunican a través de ella, el sistema se conoce como **multiprocesador con memoria compartida**.

Estos últimos, pueden clasificarse en dos enfoques principales, de acuerdo a la forma que tengan de asignar procesos a los procesadores:

- **Arquitectura maestro/esclavo:** el núcleo del SO siempre se ejecuta en un determinado procesador, y el resto de procesadores sólo ejecutan programas de usuario y algunas utilidades del SO. El *maestro* es quien planifica los procesos y los hilos, mientras que los *esclavos* piden servicios al maestro y esperan a que se realicen. En este enfoque, un procesador tiene el control de toda la memoria y recursos de E/S. Entre sus desventajas encontramos: un fallo en el maestro puede producir la caída de todo el sistema; el maestro puede convertirse en un cuello de botella respecto al rendimiento, ya que es el único responsable de toda la planificación y gestión de procesos.
- **SMP (Multiprocesamiento simétrico/Symmetric Multiprocessor):** el núcleo puede ejecutar en cualquier procesador, y normalmente cada uno de ellos realiza su propia planificación de procesos e hilos. En este enfoque, existen múltiples procesadores y cada uno con sus elementos, además de tener acceso a una memoria principal compartida y dispositivos de E/S. Las principales características de diseño de los SPM son:
 - *Procesos o hilos simultáneos concurrentes:* varios procesadores ejecutan el mismo código del núcleo simultáneamente.
 - *Planificación:* se realiza en cada uno de los procesadores, es decir, cada uno se "auto-planifica".
 - *Sincronización:* permite acceder a espacios de direcciones o recursos de E/S compartidos.
 - *Gestión de memoria:* necesaria para asegurar la consistencia a la hora de compartir información entre los procesadores.

- *Fiabilidad y tolerancia a fallos*: el SO no se debe degradar en caso de fallo de un procesador, por lo que se deben reestructurar rápidamente a la hora de que se sucedan. El sistema está controlado por un SO integrado, que proporciona la interacción entre los procesadores y sus programas en los niveles de trabajo, tareas, archivos y datos.

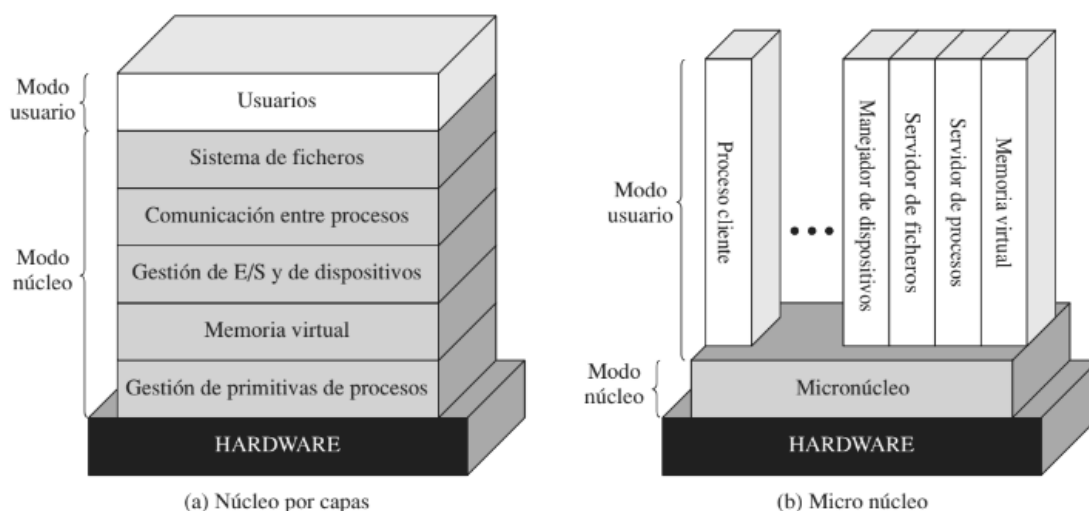
Entre las ventajas de SMP encontramos: la ejecución de procesos o hilos se puede realizar en paralelo; disponibilidad (como todos los procesadores pueden realizar las mismas funciones, un fallo en uno de ellos no hará que la computadora deje de funcionar); crecimiento incremental (se puede aumentar las prestaciones si se añaden más procesadores).

AGRUPACIONES - CLUSTERS

Un **cluster** es un conjunto de computadoras interconectadas o nodos que trabajan juntas, creando la ilusión de ser una única máquina. Estos nodos, están conectados por rutas prefijadas o redes de alta velocidad, y proporcionan escalabilidad, alto rendimiento y alta disponibilidad.

EVOLUCIÓN DE LA ARQUITECTURA DE LOS SISTEMAS OPERATIVOS

- *SO Monolíticos*: fueron los primeros SO, donde prácticamente se podía llamar a un procedimiento desde cualquier otro, demostrando su gran faltante de estructura a medida que los sistemas crecían.
- *SO por capas*: sus funciones se organizan jerárquicamente y sólo hay interacción entre las capas adyacentes. En este enfoque, la mayoría de capas se ejecutan en modo núcleo. Nuevamente, entre sus desventajas, tenemos que cada capa posee demasiadas funcionalidades, y cualquier cambio o fallo en una de estas trae consigo numerosos efectos,
- *Arquitectura Micronúcleo*: tiene como base la idea que solamente las funciones absolutamente esenciales del SO estén en el núcleo, mientras que las demás se construyen en el micronúcleo pero se ejecutan en modo usuario.



Es una arquitectura cliente-servidor, donde todas las funciones externas al micronúcleo se implementan como procesos servidores, y se comunica mediante el paso de mensajes. En esta, es el núcleo el que permite la comunicación entre servidores, administra los mensajes y protege el intercambio de mensajes.

Entre sus **ventajas** encontramos:

- *Uniformidad de interfaces*: como todos los servicios se proporcionan a través del paso de mensajes, no se necesita diferenciar entre servicios a nivel de núcleo y usuario.
- *Extensibilidad*: permite agregar nuevos servicios y realizar múltiples servicios en la misma área funcional.
- *Flexibilidad*: se adapta a las necesidades del usuario.
- *Portabilidad*.
- *Fiabilidad*: el micronúcleo puede probarse rigurosamente, de tal forma que se encuentre casi exento de fallos.
- *Soporte de sistemas distribuidos*: ya que es una arquitectura cliente/servidor.
- *Soporte para SO orientados a objetos*.

Dentro del diseño general del micronúcleo, tenemos algunas **funciones** indispensables: *gestión de memoria a bajo nivel* (traduce la dirección de página virtual en marco físico); *comunicación entre procesos* (Interprocess Communication, la comunicación entre procesos o hilos se hace mediante mensajes, además, se asocian puertos o colas de mensajes a cada proceso); *gestión de E/S e interrupciones* (esta arquitectura permite manejar las interrupciones como mensajes, e incluir los puertos de E/S en los espacios de direcciones).

COMUNICACIÓN ENTRE PROCESOS

Con frecuencia, los procesos necesitan comunicarse con otros procesos. Esto puede ser, entre otros casos, por dependencia o cuando usan los mismos recursos. Por otro lado, los procesos que trabajan de forma conjunta, comparten memoria para leer y escribir datos.

PROCESOS CONCURRENTES

Los procesos se pueden dividir en secuenciales y concurrentes. Los secuenciales, son aquellos en donde se ejecutan uno a uno, y los concurrentes son aquellos que aceptan que, en un mismo periodo de tiempo, se ejecuten varios procesos (hay superposición temporal). Decimos entonces que dos procesos son concurrentes si la primera instrucción de uno de ellos comienza antes que se complete la última instrucción del otro. Estos últimos, se pueden dividir a su vez en:

- Procesos disjuntos o NO interactuantes: operan sobre conjuntos de variables disjuntas o distintas. Si bien estos pueden leer sobre lo mismo, cada uno hace cosas distintas.

- Procesos interactuantes: son dependientes del tiempo, y la ejecución de un proceso depende del otro. En este caso, los procesos pueden operar o ejecutar una lectura o escritura en un recurso compartido, lo cual puede traer inconvenientes.

CONDICIONES DE COMPETENCIA

Las condiciones de competencia o carrera, es la situación en la cual dos o más procesos leen o escriben en lugares compartidos y el resultado final depende de quién ejecuta qué y en qué momento lo hace. La forma para evitar este problema, es mediante la aplicación de la **exclusión mutua**, asegurando que si un proceso está usando una variable o archivo compartido, los demás procesos se excluirán de hacer lo mismo (prohíbe que más de un proceso leo o escriba datos al mismo tiempo). Aquellas partes de la memoria en las que se accede a la memoria compartida, se denomina **región o sección crítica**. Entonces, si de alguna forma se pudiera ordenar los datos para que nunca se encuentren en dichas regiones, se podría evitar las condiciones de carrera. Para tener una buena solución a las carreras, se deben cumplir las siguientes condiciones:

1. No pueden haber dos procesos simultáneamente dentro de sus regiones críticas.
2. No pueden hacerse suposiciones acerca de la velocidad o número de CPUs.
3. Ningún proceso que se ejecute fuera de su región crítica puede bloquear a otros procesos.
4. Ningún proceso tiene que esperar para siempre para entrar a su región crítica.

Retomando con la exclusión mutua, decimos que esta puede ser:

- **Exclusión mutua con espera ocupada:** La espera ocupada es la acción de evaluar en forma continua una variable hasta que aparezca cierto valor. Solo se usa cuándo hay una expectativa razonable de que la espera será corta, ya que sino se pierde mucho tiempo de la CPU. Básicamente, optimiza el uso de la CPU, evitando constantemente la posibilidad de ingresar a la sección crítica. Para esto, usa dos primitivas de sincronización: *SLEEP* (llamada al sistema que produce que el proceso se bloquee o desactive, hasta que otro proceso lo despierte) y *WAKEUP* (permite activar un proceso que estaba suspendido o bloqueado, además tiene un parámetro que es el proceso que se va a despertar o activar). Está formada por cinco mecanismos:
 1. *Deshabilitando interrupciones:* cada proceso deshabilita todas las interrupciones después de entrar a su región crítica (imposibilitando las interrupciones de reloj), y las rehabilita cuando salga.
 2. *Variables candado:* solución de software que tiene un valor inicial o candado en cero, y cuando quiere entrar a la sección crítica evalúa dicho valor. Si al hacerlo se encuentra en cero, se fija en uno y entra a la región, y en caso contrario, se espera a que se haga cero.
 3. *Alternancia estricta:* requiere que los procesos se alternen de manera estricta al entrar en sus regiones críticas, logrando entre otras cosas, impedir las condiciones de carrera.

4. *Solución de Peterson*: tiene una base en las ideas de las variables de candado, y sin la necesidad de la alternancia estricta.
5. *La instrucción TSL*: nuevamente, este mecanismo toma las bases de las variables de candado, pero lleva su solución a nivel de hardware.

- ***Exclusión mutua sin espera ocupada***: Dentro de los mecanismos que la forman encontramos:

- ***Semáforos***: variable entera que cuenta la cantidad de “wakeup” almacenados para su uso futuro. Vale cero en caso de que no se hayan guardado señales de despertar, o un valor positivo si tiene pendientes una o más señales de despertar. Cada proceso tiene su propio semáforo. Tiene dos operaciones: *DOWN* (comprueba si el valor es mayor a cero, y en caso de serlo, disminuye el valor y continua) y *UP* (incrementa el valor del semáforo direccionado). Estas operaciones se denominan *acciones atómicas* y se encargan de evaluar el valor de los semáforos, modificarlo y ejecutar un sleep de forma conjunta e indivisible, además estas acciones impiden las interrupciones, problemas de sincronización y las condiciones de carrera.

Los semáforos pueden ser binarios (garantizando la exclusión mutua) o de sincronización.

- ***Monitores***: conjunto de procedimientos, variables y estructuras de datos agrupados en un módulo o paquete. Son una primitiva de sincronización de alto nivel, que permite un único proceso activo por monitor en un determinado momento.
- ***Mensajes***: método de comunicación entre procesos, usado en entornos centralizados y distribuidos. Un mensaje en sí, es la información que se intercambia entre un proceso emisor y un proceso receptor. El mensaje incluye una cabecera que identifica a los procesos remitentes y receptor, y un cuerpo que contiene todos los datos (origen, destino, número de secuencia, CRC o código de redundancia mínima y longitud).

Según la longitud de los mensajes, estos pueden ser *fijos*, en caso de tener la misma longitud, o *variables* si sus datos y cabecera pueden cambiar de longitud.

En lo que refiere a la transferencia de los mensajes, tenemos dos primitivas de sincronización: *SEND (destino, mensaje)* y *RECEIVE (origen, mensaje)*. Con tal de protegerse de la pérdida de mensajes, el emisor y receptor acuerdan en enviar un nuevo mensaje una vez se recibió el mensaje enviado originalmente. Además, en caso de que el emisor no reciba el mensaje en un cierto tiempo, entonces vuelve a transmitir el mensaje. Estas reglas se denominan *reconocimiento* o *acknowledgment*. Otra de las cuestiones que tienen que ver con las soluciones a los problemas, tiene que ver con la *autenticación*. Esta permite identificar tanto al emisor como al receptor, con tal de saber si se está comunicando con quien realmente se

quería en un inicio. Por último, se debe tener en cuenta el *rendimiento*, es decir, la cantidad de mensajes que se envían por unidad de tiempo. Esto se expresa en *bps* (bits por segundo, donde Kbps = 10^3 bps; Mbps = 10^6 bps; Gbps = 10^9 bps).

PROBLEMA DEL PRODUCTOR Y CONSUMIDOR

Problema en el que dos procesos comparten un búfer común y de tamaño fijo. Uno de los procesos, el productor, coloca información en el búfer y el otro, el consumidor, la saca. El problema está cuando el productor desea agregar un nuevo elemento al búfer pero este ya está lleno. Para solucionarlo, habría que poner a dormir al productor y que se despierte cuando el consumidor haya sacado elementos del búfer. En contraposición tenemos al consumidor, que si encuentra el búfer vacío, va a hacer lo mismo que el productor cuando este está lleno.

Debemos llevar la cuenta de los elementos del búfer en una variable. Si el número de máximo de elementos que puede contener es N, el código del productor comprueba que la variable sea igual a N, y si lo es duerme, y si no agrega un elemento e incrementa la variable. Por el lado del consumidor, este evalúa la variable para ver si vale cero. Si lo es duerme, y si no quita un elemento y disminuye el contador de la variable. Además, tanto el consumidor como el productor, son los encargados de despertarse uno al otro en caso de ser necesario.

Puede pasar que el búfer está vacío y el consumidor acaba de leer la variable para ver si es cero, y en ese momento, el planificador decide detener al consumidor de forma temporal y empieza a ejecutar el productor, quien inserta un elemento e incrementa la cuenta, la cual ahora vale uno. Ahora bien, cuando el productor llama al consumidor para que se despierte, este último todavía no estaba lógicamente dormido, y finalmente si se pone a hacerlo, por lo que la señal se pierde, y el consumidor no irá a sacar el elemento. Esto provoca que el productor llene el búfer hasta el punto en el que se duerme, con la salvedad de que esta vez no se presentará el consumidor para sacar los elementos, ya que ambos estarán permanentemente dormidos.

TIPOS DE COMUNICACIÓN

- **Sincrónica:** el proceso emisor y receptor deben acordar previamente el intercambio de información.
- **Asincrónica:** usan el buffer o almacenamiento inmediato entre los procesos emisor y receptor.

PLANIFICACIÓN DE PROCESOS

Como ya sabemos, en un sistema multiprogramado, hay múltiples procesos concurrentes en memoria principal que pueden estar usando un procesador o pueden estar esperando el suceso de algún evento. Mientras tanto, el o los procesadores, se mantienen ejecutando un proceso mientras los demás esperan. *El objetivo de la planificación de procesos, es asignar procesos a ser ejecutados por el procesador a lo largo del tiempo, de*

forma que se cumplan los objetivos del sistema (tiempo de respuesta, rendimiento y eficiencia de procesador). La planificación afecta al rendimiento del sistema porque determina qué proceso esperará y cuál progresará.

Los **tipos de planificación** son:

- *Planificación a largo plazo*: determina qué programas se admiten en el sistema para su procesamiento, controlando el grado de multiprogramación. Una vez admitido, un programa de usuario se convierte en un proceso y se añade a la cola de planificador a corto plazo. Es el planificador menos frecuente.



- *Planificación a medio plazo*: es parte de la función de intercambio o swapping. Su función es de gestora del grado de multiprogramación. Esta se encarga de decidir en qué momento se deben suspender y activar los procesos.



- *Planificación a corto plazo*: selecciona el proceso que está en estado Listo. Se invoca siempre que ocurre un evento que puede conllevar el bloqueo del proceso actual y que puede proporcionar la oportunidad de expulsar el proceso actualmente en ejecución a favor de otro. Estos eventos pueden ser: interrupciones de reloj o de E/S, llamadas del sistema y señales.





CRITERIOS DE PLANIFICACIÓN

- *Rendimiento*: debe intentar maximizar el número de procesos completados por unidad de tiempo.
- *Utilización de procesador*: porcentaje de tiempo que el procesador está ocupado
- *Equidad*: dar a cada proceso una porción equitativa del tiempo de CPU, ya que ningún proceso debe sufrir de inanición.
- *Cumplimiento de políticas (imposición de prioridades)*: cuando se asignan prioridades a los procesos, la política del planificador debe favorecer a los procesos de mayor prioridad.
- *Equilibrado de recursos*: la política del planificador debería mantener ocupados los recursos del sistema.
- *Rendimiento (respecto al usuario)*: número de trabajos que el sistema termina por hora. Mientras más rendimiento, mejor.
- *Tiempo de retorno*: promedio de tiempo que transcurre entre que comienza el trabajo y termina, es decir, da el tiempo promedio que espera el usuario en recibir la salida. Mientras menos tiempo, mejor.
- *Previsibilidad*: un trabajo debería ejecutarse al mismo tiempo y con el mismo coste a pesar de la carga del sistema.

TIPOS DE ALGORITMOS DE PLANIFICACIÓN

- **Algoritmos NO apropiativos (sin expulsión)**: selecciona un proceso para ejecutarlo y después deja que se ejecute hasta que el mismo se bloquea o hasta que libera la CPU de forma voluntaria. No se pueden suspender de manera forzada, debido a que no se toman decisiones de planificación durante las interrupciones de reloj. Al finalizar el procesamiento de la interrupción de reloj, se reanuda la ejecución del proceso que estaba antes de la misma o se ejecuta algún proceso de mayor prioridad que se encuentre en la cola.
- **Algoritmo apropiativo (con expulsión)**: selecciona un proceso y deja que se ejecute durante un tiempo establecido o quantum. Si al terminar el tiempo sigue con la

ejecución, se suspende y el planificador selecciona, en el caso que haya, otro proceso para ejecutarlo.

CATEGORÍAS DE LOS ALGORITMOS DE PLANIFICACIÓN

Los algoritmos de planificación varían para cada entorno, ya que no comparten objetivos y deben optimizar partes distintas de cada sistema. Los tres entornos más importantes son:

1. **Procesamiento por Lotes:** estos aceptan algoritmos no apropiativos o apropiativos con largos periodos para cada proceso. Este método reduce la conmutación de procesos, mejorando el rendimiento. Dentro de este, los más importantes son:
 - *Primero en entrar, primero en salir:* la CPU se asigna a los procesos en el orden en el que la solicitan. Hay una única cola de procesos listos, y a medida que se ingresen trabajos nuevos, o los procesos pasen del estado Bloqueado a Listo, se colocan al final de la cola. Si el proceso en ejecución se bloquea, el primer proceso en la cola se ejecuta a continuación. Es el algoritmo más simple, además de ser sencillo de entender y programar.
 - *El trabajo más corto primero:* se presenta cuando hay varios trabajos de igual importancia esperando a ser iniciados en la cola de entrada, y se conoce de antemano los tiempos de ejecución de cada uno, entonces se elige al trabajo de menor tiempo de ejecución. Es importante destacar que este algoritmo es óptimo siempre que todos los trabajos estén disponibles al mismo tiempo.
 - *El menor tiempo restante a continuación:* es una versión apropiativa del algoritmo tipo “el trabajo más corto primero”, pero en el que el planificador siempre selecciona el proceso cuyo tiempo restante de ejecución es el más corto. Se debe conocer el tiempo de ejecución de antemano, ya que cuando llega un nuevo trabajo, su tiempo total se compara con el tiempo restante del proceso actual. Si el nuevo proceso requiere menos tiempo para terminar que el proceso actual, éste se suspende y el nuevo inicia.
2. **Planificación en sistemas Interactivos:** en este entorno, la apropiación es esencial para evitar que un proceso acapare la CPU y niegue el servicio a los demás. Los algoritmos más importantes dentro de esta categoría son:
 - *Planificación por turno circular o algoritmo de planificación Round Robin:* a cada proceso se le asigna un intervalo de tiempo o quantum, durante el cual puede ejecutarse.

Si se sigue ejecutando al finalizar el tiempo, la CPU se lo da a otro proceso, pero si se bloquea o termina antes de que se cumpla el tiempo, la conmutación de la CPU se realiza cuando el proceso se bloquea. Por otro lado, si hay “n” procesos en la cola de listos y el tiempo es “q”, entonces cada proceso toma “1/n” del tiempo de CPU a lo sumo unas “q” unidades de tiempo a la vez. Los procesos no esperan más que “(n-1)q” unidades de tiempo.

Si se establece un quantum demasiado corto, se producen demasiadas conmutaciones de procesos y se reduce la eficiencia de la CPU (demasiados cambios de contexto), y si el tiempo es demasiado largo, se puede producir una mala respuesta a las peticiones interactivas cortas (mayor tiempo de respuesta). Es por esto que se establece que la duración razonable de un quantum debe ser de entre 20 y 50 milisegundos (mseg).

- *Planificación por prioridad:* a cada proceso se le asigna una prioridad y el proceso ejecutable con la prioridad más alta es el que se puede ejecutar. Dicha prioridad se puede asignar en forma estática o dinámica.
- *Planificación por colas múltiples:* busca reducir el intercambio o cambio de contexto. En este enfoque, nuevamente se realiza una división por clases de prioridades, en donde los procesos de más alta clase se ejecutan durante un quantum, y así hasta agotar todos sus quantum, pasando a la siguiente jerarquía.
- *Algoritmo de planificación garantizada:* se realizan promesas o compromisos reales a los usuarios acerca del rendimiento, que luego se deben cumplir. Una de estas promesas es: si hay “n” usuarios conectados, recibirán aproximadamente “1/n” del poder de la CPU. Periódicamente, se calcula como “CPU usada/CPU prometida”, y puede dar como resultado que el proceso use más o menos CPU de lo prometido.

INTERBLOQUEOS

Un conjunto de procesos se encuentran en un interbloqueo, si cada proceso del conjunto está esperando un evento que sólo sea ocasionado por otro proceso del conjunto. Como todos los procesos están esperando, ninguno podrá devolver ningún recurso y seguirán bloqueados indefinidamente.

RECURSOS

Los elementos que son usados por un solo proceso en un momento determinado, se denominan recursos. La secuencia de eventos requerida para usar un recurso, es: solicitarlo, usarlo y liberarlo. Estos recursos se pueden dividir en:

- *Recursos no apropiativos (sin expulsión):* son aquellos que no se pueden quitar a su propietario si hacer que el cómputo falle. En este caso, el proceso se ejecuta hasta que finaliza o se bloquea.
- *Recursos apropiativos (con expulsión):* son aquellos que se pueden quitar del proceso que lo posee sin efectos dañinos. El proceso se ejecutará durante un tiempo o quantum, el cuál es controlado por interrupciones de reloj.

CONDICIONES PARA LOS INTERBLOQUEOS

Para los interbloqueos de los recursos, se deben cumplir cuatro condiciones (y en caso de que una de estas no se presente, no se presentará):

1. **Exclusión mutua:** cada recurso se asigna en un momento dado a un sólo proceso o está disponible.

2. **Contención y espera:** un proceso posee recursos y puede solicitar otros.
3. **No apropiativo:** los procesos otorgados previamente no se pueden quitar a un proceso por la fuerza, sino que debe ser liberado por el proceso que lo contiene.
4. **Espera circular:** debe haber una cadena circular de dos o más procesos, y cada uno espera un recurso contenido por el siguiente miembro de la cadena.

ESTRATEGIAS PARA ENFRENTAR LOS INTERBLOQUEOS

1. IGNORAR EL PROBLEMA - ALGORITMO DEL AVESTRUZ

Enfoque que establece que es preferible que se produzca un interbloqueo esporádicamente y no sobrecargar el sistema para tratar de evitarlo. Para esto, se debe hacer un análisis para determinar con cuánta frecuencia se produce un interbloqueo.

2. DETECCIÓN - RECUPERACIÓN

Se permite que se produzcan interbloqueos, se detectan y se toman acciones. El SO intenta detectar la ocurrencia de un interbloqueo, y en caso de hacerlo, aplica alguna técnica de recuperación.

- Detección de un interbloqueo: se realiza con uno (los más simples) con o varios recursos de cada tipo.
- Recuperación de un interbloqueo: la recuperación consiste en, tras la detección de los interbloqueos, encontrar la forma para hacer funcionar el sistema nuevamente. Las tres formas de recuperación más importantes son:

1. **Por medio de apropiación:** se le quita el recurso al proceso temporalmente y se lo asigna a otro proceso. Esta forma es de difícil implementación.
2. **A través del retroceso (rollback):** se hace que los procesos hagan puntos de comprobación o verificación en forma periódica, escribiendo su estado (qué recursos están asignados al proceso en un momento dado) en un archivo para su reinicio posterior. Para la recuperación, se revierte a un punto en el tiempo antes de que se haya adquirido ese recurso, iniciado en uno de los puntos de comprobación anterior al interbloqueo.
3. **A través de la eliminación de los procesos:** se realiza la eliminación de uno o varios procesos que estén en el ciclo.

3. EVITAR LOS INTERBLOQUEOS

En estos, es el sistema quien decide si es seguro o no otorgar un recurso, y quién establece cuidadosamente la asignación de los mismos. Cada proceso debe determinar la cantidad y tipo de recursos que utilizará. Se tienen en cuenta dos estados: el *seguro* (el sistema garantiza que todos los procesos pueden finalizar su ejecución) y el *inseguro* (el sistema no puede brindar garantía que todos los procesos finalizarán su ejecución).

4. PREVENIR LOS INTERBLOQUEOS

Es algo de esencia imposible, debido a que se requiere información sobre las peticiones futuras, las cuales no se conocen. Pero se puede llegar a prevenir en caso de que una de las condiciones para que se produzcan los interbloques sea atacada. Estas formas de ataque para cada condición, son:

- *Condición de exclusión mutua:* se delega la responsabilidad de la administración y acceso al recurso al SO a través de procesos “daemons o demonios”.
- *Condición de posesión y espera:* se puede evitar que los procesos que contiene recursos esperen por más recursos. Se puede lograr solicitando que todos los procesos soliciten todos sus recursos antes de empezar la ejecución, y en caso de que esté todo disponible, al proceso se le asignará lo que necesite y se ejecutará hasta completarse. Si uno o más recursos están ocupados, no se asignará nada, y el proceso esperará. Además, si un proceso no puede tener todos los recursos que necesita, debe liberar los que posee actualmente.
- *Condición de no apropiativo:* condición difícil de anular. Entre otras cosas, consiste en virtualizar ciertos elementos que resultan difícil de anular a la fuerza, para que sean controlados primeramente por un demonio. Es muy limitado, ya que no todos los recursos se pueden virtualizar de esta manera.
- *Condición de espera circular:* una de las formas, consiste en numerar todos los recursos, permitiendo que los procesos puedan solicitar recursos cada vez que quieran, pero todas las peticiones se realizan en orden numérico. Si un proceso solicita un recurso cuya numeración sea menor al recurso que ya posee, debe liderar este recurso.

UNIDAD N°4: ADMINISTRACIÓN DE MEMORIA

La parte del SO que administra la jerarquía de memoria (porción de memoria destinada al caché, a la memoria principal, al disco lento y al almacenamiento removible) se conoce como **administrador de memoria**. Este, administra la memoria con *eficiencia*: lleva el registro de las partes de memoria en uso, asigna memoria a los procesos cuando se necesitan y los desasigna cuando terminan.

Dentro de las diferentes técnicas de administración de memoria encontramos: *Monoprogramación*, *Multiprogramación* (con particiones fijas, con particiones variables) y *Memoria virtual* (paginación y segmentación).

MONOPROGRAMACIÓN

Esta técnica solo permite ejecutar un proceso a la vez, y tiene un solo programa de usuario en memoria. Esto conlleva bajo uso de la memoria (solo un programa en memoria) y del procesador (solo un programa para ejecutar). En la memoria, además del programa, permite almacenar al SO con sus funcionalidades principales.

MULTIPROGRAMACIÓN CON PARTICIONES FIJAS

Divide la memoria en “n” particiones de diferentes tamaños, y en cada una de estas secciones fijas, se carga un proceso y permanece hasta que termina. Las particiones pueden ser:

- *Con colas independientes*: cada una de las particiones tiene un espacio determinado y es donde se instalan los procesos. Al tener un espacio fijo, son muy propensas a la fragmentación externa (cuando a un espacio no se le puede meter dentro un proceso más grande que el mismo).
- *Con cola única*: busca en la cola el proceso más grande que se ajuste a la partición recién liberada, y es allí donde los coloca. Tiene como inconveniente, que los procesos más pequeños tienden a no ser seleccionados, ya que busca principalmente los procesos con mayor espacio. Pero para evitar este problema, se ajusta la cantidad de veces que se puede excluir un proceso, dejando entonces a los procesos más viejos delante de los más nuevos en ese punto.

ABSTRACCIÓN DE MEMORIA

1. **Espacio de direcciones**: conjunto de direcciones que puede usar un proceso para direccionar la memoria. Cada proceso tiene su propio espacio de direcciones privado, independiente del resto de procesos, el cual es incorporado junto a los demás recursos a la hora de la creación del proceso.
2. **Reasignación o reubicación**: en el registro base se cargan las direcciones físicas donde empieza el programa en memoria. Cuando un proceso hace referencia a la memoria, el hardware de la CPU suma automáticamente el valor de dicho registro

base a la dirección generada por el proceso antes de enviar la dirección al bus de memoria.

3. **Protección:** se controla que el proceso sólo dirija su propio espacio de memoria. En el registro límite se carga la longitud del programa. Se protegen las referencias a memoria fuera de rango, es decir que la dirección relativa del mismo no debe superar la suma de los valores del registro base más la del registro límite, en caso contrario, se genera un fallo y no se permite el acceso a esa zona de memoria.
4. **Intercambio:** se lleva el proceso completo a memoria, se ejecuta un tiempo y se baja a disco. Los procesos inactivos son almacenados en disco, para liberar memoria mientras no se estén ejecutando. Cuando un proceso se vuelve a cargar en memoria RAM, puede ubicarse en otro espacio de direcciones diferente.

MULTIPROGRAMACIÓN CON PARTICIONES VARIABLES

En esta técnica, la porción de memoria que la forma es variable. Mediante el *intercambio*, se lleva el proceso completo a memoria y se baja al disco. Todos los procesos inactivos se almacenan en el disco o memoria secundaria. Para hacerle frente a la fragmentación externa, este tipo de multiprogramación permite reordenar la ubicación de los procesos, con tal de disminuir los espacios vacíos que la producen, todo a un importante costo.

En esta, el número y tamaño de los procesos varía en forma dinámica, y las particiones se crean a medida que llegan los procesos, y se adaptan a estos. Para la administración de los espacios de memoria en estos casos, se pueden usar las siguientes técnicas:

- *Mapa de bits:* la memoria se divide en *unidades de asignación* de cierto tamaño, y se le da a cada una un bit (0, 1), que indican si se encuentran ocupadas o no. Si la unidad de asignación es pequeña, el mapa de bits es grande, y viceversa. Esto significa que el tamaño del mapa de bits depende del tamaño de la memoria y el tamaño de la unidad de asignación. Cuando se debe llevar un proceso a memoria, el administrador busca en el mapa una serie de bits de la misma longitud que el proceso, y con el valor asignado "0" (desocupado). Este mecanismo de búsqueda se hace de forma lenta, debido a que la serie puede cruzar los límites entre las palabras del mapa.
- *Lista enlazada o ligada:* se realiza sobre segmentos de memoria asignados y libres, en donde un segmento contiene un proceso o es un hueco vacío entre dos procesos. Cuando los procesos y huecos se mantienen en la lista ordenada por dirección, se pueden aplicar algoritmos para asignar memoria a un proceso creado. Estos pueden ser:
 - *Primer ajuste:* es el más simple, donde el administrador explora la lista de segmentos hasta encontrar un hueco que sea lo bastante grande (donde quepa el proceso).

- *Siguiente ajuste:* igual al anterior, continuando la búsqueda en la lista a partir de dónde quedó la última vez.
- *Mejor ajuste:* busca en toda la lista, de principio a fin y toma el hueco más pequeño que sea adecuado (el más pequeño donde quepa).
- *Peor ajuste:* toma el hueco más grande disponible donde quepa el proceso.
- *Ajuste rápido:* hace una lista ordenada de huecos según su tamaño, manteniendo listas para algunos de los tamaños más comunes solicitados.

MEMORIA VIRTUAL

La MV es una técnica de administración de memoria en la cual el SO dispone tanto para las aplicaciones de usuario como para el código del SO, de mayor cantidad de memoria que la que posee disponible físicamente.

MV = RAM + ÁREA DE SWAP del almacenamiento secundario

El área de Swap puede aumentar su tamaño de distintas formas, siendo el swap una memoria de tipo volátil.

Los *inconvenientes* que busca solucionar la MV son: un proceso debe estar cargado en RAM todo junto en direcciones de memoria de forma contigua; no se puede ejecutar un proceso de mayor tamaño que la RAM; el grado de multiprogramación es afectado tanto por el tamaño del proceso como el de la RAM.

Dentro de las *características* de la MV encontramos:

- Mediante esta técnica, el SO reserva un espacio en el disco para poder usarlo como si fuese memoria RAM.
- El sistema crea, para cada proceso, un espacio de direcciones virtual en el disco. El espacio de direcciones está representado por números virtuales.
- Permite ejecutar programas cuyo tamaño sea mayor a la memoria física o RAM disponible.
- El programa se divide en partes, las cuales se pueden almacenar en disco y en memoria (Swap y RAM).
- Se aumenta considerablemente el grado de multiprogramación.
- Existen dos técnicas de implementación: paginación y segmentación.

PAGINACIÓN

Es el espacio de direcciones virtuales generadas por cada programa. Cada programa posee su propio espacio de direcciones virtuales, además de dividirse en partes del mismo tamaño denominadas "*páginas*", las cuales se almacenan en disco y se van llevando a memoria a medida que se necesitan. Las unidades correspondientes en la memoria física (RAM) se llaman "*marcos de página*". Tanto el tamaño de las páginas como el de los marcos de páginas, son iguales.

PÁGINA - virtual (posee el número de marco)

MARCO DE PÁGINA - físico (número en donde se encuentran en la RAM)

Los marcos de página no requieren estar asociados a la memoria RAM en forma contigua o adyacente. Por otro lado, las páginas deben estar dispuestas en forma continua (los marcos no necesariamente).

TRADUCCIÓN DE DIRECCIONES

Las direcciones virtuales no van directamente al bus de memoria (canales de comunicación), sino que existe la necesidad de traducir las direcciones virtuales en direcciones físicas, o bien, traducir los números de página al marco de página. La encargada de esto es la *MMU* (Memory Management Unit - Unidad de Administración de Memoria), que se encuentra formada por un conjunto de chips asociados a direcciones virtuales con direcciones físicas de la memoria. En esta, la dirección está formada por un conjunto de bits que hacen al número de página y al desplazamiento o ajuste.

FALLO DE PÁGINA

Existe cuando un programa intenta usar una página que no está cargada en la RAM. La MMU detecta que la página no está asociada o no está en la RAM, y hace que la CPU genere un *Trap* al SO. Este consiste en liberar espacio de la RAM, bajando alguna página a disco, y cargando desde el disco a la RAM la página solicitada, asociándose a la tabla de páginas, para finalmente continuar con la ejecución.

TABLA DE PÁGINA

Es una función donde el número de página virtual es un argumento y el número del marco físico es un resultado. Cada proceso tiene su propia tabla de páginas, y su tamaño depende del tamaño del proceso y del tamaño de las páginas.

Cada entrada de la tabla posee los siguientes elementos:

- *Número de página.*
- *Número de marco de página.*
- *Bit P/A (presente/ausente):* indica si la página está en memoria o en disco. Si es 1 la entrada es válida y se puede usar, y si es 0, la página virtual a que pertenece la entrada no se encuentra en ese momento en memoria.
- *Bit R (referencia):* se activa cada vez que se accede a una página, tanto para lectura como escritura. Permite ayudar al SO a determinar qué página desalojar de ser necesario (fallo de página), en base a ver cuáles son las páginas que no se están usando (es decir, las más indicadas) y cuáles sí.
- *Bit M (modificación):* se activa cuando se modifica la página. Ayuda al SO a conocer las páginas que han sido modificadas, a fin de volverlas a escribir en el disco.
- *Bit de protección:* indica que tipo de acceso está permitido. Puede estar formado por dos bits o tres, en el primer caso, el 0 representa el acceso de lectura/escritura, y el 1 acceso de solo lectura.

- *Bit de caché*: permite deshabilitar el uso del caché para la página.

Número de PÁGINA	Número de MARCO de Página	Bit P/A	Bit R	Bit M	Bits de Protección	Bit de caché
---------------------	------------------------------	---------	-------	-------	-----------------------	--------------

TRANSLATION LOOKASIDE BUFFER (TLB)

El *TLB* o **memoria asociativa** es un dispositivo de hardware para asociar direcciones virtuales a direcciones físicas sin pasar por la tabla de páginas. Tiene como objetivo acelerar el proceso de traducción de direcciones virtuales en direcciones físicas. Está ubicado en el interior de la MMU, y está formado por un número de entradas (hasta 64) con los datos de la tabla de páginas.

Cuando llega una dirección virtual de la MMU, el hardware verifica primero si el número de páginas virtuales está en el TLB, comparando todas las entradas en paralelo e intenta hacer la traducción. Si no está en el TLB, hace una consulta normal a la tabla de páginas y actualiza el TLB, habiendo previamente liberado una entrada del TLB.

TABLA DE PÁGINAS MULTINIVEL

Estas tablas tienen como objetivo evitar mantener en memoria todas las tablas de páginas todo el tiempo, y más aún aquellas que no se necesitan.

Básicamente, sirve a modo de solución para todos los procesos que necesitan una tabla de páginas, y tengan un alto nivel de ocupación en memoria por la extensión de las mismas.

TABLA DE PÁGINAS INVERTIDA

Sirve a modo de solución para aquellos procesos con una gran cantidad de entradas, que no son capaces de ser procesadas en un tiempo considerable. Entonces, las tablas de páginas invertidas, están formadas por una entrada por cada marco de página de la memoria real (RAM), en vez de tener una entrada por página de espacios de direcciones virtuales. Así, cada entrada lleva el registro de que proceso y página virtual se encuentra en el marco de página.

Esta tabla permite el ahorro de gran cantidad de espacio de memoria RAM. Ahora bien, tiene como desventaja la complejidad en la traducción de dirección virtual a física (pero se soluciona con la utilización de TLB y la tabla de hash).

ALGORITMO DE REEMPLAZO DE PÁGINAS

Estos algoritmos existen para que, cuando se produce un fallo de página, estando la memoria llena, se desaloje una página de RAM para hacer espacio para la nueva página. Para esto, se debe analizar el bit "M" de la página a desalojar. Si este bit es igual a 1, se guardan los cambios en el disco, y si es 0 se libera la zona de memoria. Además, se debe analizar qué página conviene extraer de la memoria.

Los principales algoritmos de reemplazo de páginas son:

1. **Algoritmo de reemplazo de páginas óptimo:** es fácil de describir pero imposible de implementar. Al momento que ocurre el fallo de página, de un conjunto de páginas en memoria, una se referenciará en la siguiente instrucción. Así, cada página se puede etiquetar con el número de instrucciones que se ejecutarán antes de que se haga referencia por primera vez a esa página. El algoritmo establece que la página con la etiqueta más alta debe eliminarse.
2. **Algoritmo de reemplazo de páginas no usadas recientemente (NRU - No Recently Use):** elimina una página al azar de la clase de menor numeración que no esté vacía. Se establecen dos bits asociados a cada página: bit R (cuando se referencia a una página) y bit M (cuando se escribe o modifica una página). Estos bits se actualizan en cada referencia a la memoria, por lo que se establecen mediante hardware. Cuando ocurre un fallo de página, el SO inspecciona todas las páginas y las divide en cuatro categorías en base a los valores actuales de sus bits M y R:
 - Clase 0: no se referenció, no se modificó.
 - Clase 1: no se referenció, se modificó.
 - Clase 2: se referenció, no se modificó.
 - Clase 3: se referenció, se modificó.
3. **Algoritmo de reemplazo de páginas FIFO:** el SO mantiene una lista de todas las páginas actualmente en memoria, y se las extrae de acuerdo al orden de llegada de las mismas (la más reciente al final, la más antigua al frente). Si ocurre un fallo de página, se retira la página más antigua.
4. **Algoritmo de reemplazo de páginas de segunda oportunidad:** Sirve como solución al algoritmo FIFO. Antes de extraer la página, se analiza el bit de referencia (R) de la misma. Si este es 0 se retira la página de memoria, y si es 1 se limpia el bit R y la página se coloca al final de la cola. Este algoritmo permite tener en cuenta y mantener en memoria RAM a las páginas de mucho uso.
5. **Algoritmo de reemplazo de páginas del reloj:** mantiene todos los marcos de página en una lista circular en forma de reloj, donde la manecilla apunta a la página más antigua. Funciona igual que el algoritmo de segunda oportunidad, solo que cambia su forma de implementación física.

MODELO CONJUNTO DE TRABAJO

Cuando se inicia la ejecución de un proceso se deben cargar sus páginas en memoria RAM, lo que produce una gran cantidad de fallos de página, ya que el SO tiene que traer la página que contiene la instrucción. Este proceso de carga de páginas en la memoria RAM solo cuando se las necesita se conoce como **paginación por demanda**.

Los procesos poseen una **localidad de referencia** que permite, durante su ejecución, hacer referencia a solo una pequeña fracción de sus páginas.

El conjunto de páginas que usa un proceso en un momento dado se conoce como su **conjunto de trabajo**, y permite no producir demasiados fallos de página. Por otro lado,

cuando se producen demasiados fallos de página en relación a la cantidad de instrucciones que se ejecutan, se presenta lo que se conoce como **sobrepaginado** o **thrashing**.

A diferencia de la paginación por demanda, existe la **prepaginación**, que permite la carga de páginas de un proceso en RAM antes que comience la ejecución del mismo. Esta forma de paginación está incluida en lo que se conoce como “**modelo de conjunto de trabajo**”, el cuál tiene como objetivo reducir los fallos de página. Para que se pueda implementar, es necesario que el SO lleve cuenta de cuáles páginas están en el conjunto de trabajo de cada proceso, y este a su vez, cambia durante su ejecución (aumentando o disminuyendo su tamaño).

POLÍTICAS DE ASIGNACIÓN LOCAL VS ASIGNACIÓN GLOBAL

- **Políticas de asignación Local:** selecciona una página del mismo proceso que ocasionó el fallo de página. Asigna una cantidad fija de memoria a cada proceso.

Diferencias	Políticas de asignación	
	LOCAL	GLOBAL
Selección de páginas	Selecciona una página del mismo proceso que ocasionó el fallo de página.	Se selecciona una página entre todas las páginas de la memoria.
Asignación de memoria	Asigna una cantidad fija de memoria a cada proceso.	Se asigna memoria a cada proceso en forma dinámica.
Funcionamiento	Si el proceso A ocasiona el fallo de página, busca entre sus páginas la que posea menor edad.	Si el proceso A ocasiona el fallo de página, se busca entre las páginas de todos los procesos de la memoria, la que posea menor edad.
Ventajas	Simplicidad.	Funciona mejor porque se adapta a los cambios de tamaño del conjunto de trabajo.
Desventajas	- Si el conjunto de trabajo crece, se produce sobrepaginación. - Si el conjunto de trabajos se reduce, se desaprovecha la memoria.	El sistema debe decidir constantemente cuánta memoria asignarle a cada proceso.
¿Cuál es mejor?	En general, los algoritmos globales funcionan mejor, en especial cuando el tamaño del conjunto de trabajo puede variar durante el tiempo de vida de un proceso.	

TAMAÑO DE PÁGINA

Es un parámetro que el SO puede elegir. Esta puede influir en dos casos: si la página es grande se produce fragmentación interna en la última página, y si la página es muy pequeña, se aprovecha mucho mejor el uso de la memoria reduciendo la fragmentación interna. Recordemos que la fragmentación interna es el espacio desperdiciado por quedar vacío en las páginas.

ASPECTOS DE IMPLANTACIÓN

La paginación funciona mejor cuando hay suficientes marcos libres en el momento en el cual se produce un fallo de página. Existe un **demonio de paginación** que es un proceso en segundo plano que garantiza la existencia de suficientes marcos libres en memoria RAM. Este permanece inactivo por un tiempo, luego se activa y, si es necesario, libera la memoria, garantizando una determinada cantidad de marcos libres.

MANEJO DE FALLOS DE PÁGINAS

La secuencia de eventos es la siguiente:

1. El hardware hace un trap al kernel, guardando el contador de programa en la pila.
2. El SO detecta un fallo de página, es decir, descubre qué página virtual se necesita y comprueba los bits de protección.
3. Se verifica si existen marcos disponibles, de lo contrario se ejecuta el algoritmo de reemplazo de páginas. Si la página a extraer posee el bit M en 1, se graba en disco.
4. Se realiza una conmutación de contexto, suspendiendo el proceso que produjo el fallo de página.
5. Se busca la página solicitada en el disco.
6. Cuando la página está en memoria RAM, se actualiza la tabla de páginas.
7. Se continúa con la ejecución del proceso que originó el fallo, recuperando los registros y el valor del contador del programa.

BLOQUEO DE PÁGINAS EN MEMORIA

Tiene como objetivo no eliminar páginas que están realizando operaciones de E/S. Si el algoritmo de paginación es global, hay una probabilidad de que se pueda extraer una página que esté relacionada con una operación de E/S, por lo que se debe bloquear o cerrar las páginas afectadas a operaciones de E/S.

ALMACENAMIENTO DE RESPALDO

La forma más simple de asignar espacio de página en el disco es tener una partición de intercambio especial en el disco o bien, separada del SO. El área de intercambio o de Swap, es el área del disco (archivo o paginación) donde se almacena la imagen de los procesos que se están ejecutando. Cuando se usa la técnica de memoria virtual, se necesita reservar un espacio de disco para el intercambio de procesos y páginas, ahí es donde aparece el área de swap.

En la tabla de procesos se registra el área de intercambio en el disco de cada proceso que se está ejecutando. Windows usa el método de pagefile (archivo de paginación), y Linux usa el de partición de swap.

SEGMENTACIÓN

Es una técnica de administración de memoria virtual en la cual el programa se divide en partes de diferentes tamaños denominadas segmentos. Cada uno de estos posee su propio espacio de direccionamiento virtual y se procesa en forma independiente. Como cada uno de estos segmentos es un espacio de direcciones separado, estos pueden crecer o decrecer de manera independiente sin afectar a los otros. Tiende a generar *fragmentación externa*, debido a que los procesos asignados han ocupado posiciones no contiguas de memoria, dejando demasiados bloques libres de pequeño tamaño donde no caben nuevos procesos.

Entre sus ventajas tenemos:

- Cada segmento tiene un tipo de datos distinto.
- Facilita la comparación de procedimientos o datos entre varios procesos.
- Cada segmento puede crecer o decrecer de forma independiente.
- Cada segmento puede tener un tipo de protección diferente.
- El programador es consciente de que se está usando la técnica de segmentación.

TRADUCCIÓN DE DIRECCIONES

El usuario proporciona dos valores: un número de segmento (se usa para encontrar el descriptor de segmentos, donde se indica si está o no en memoria principal) y uno de desplazamiento (para obtener la dirección de memoria principal donde se encuentra, y debe estar entre 0 y el límite del segmento). Con esto, se cumple que:

$$\text{Dirección lógica} = \text{Número de segmento} + \text{Desplazamiento dentro del segmento}$$

Dentro de una tabla de segmentos, se mapean las direcciones bidimensionales definidas por el usuario sobre las direcciones físicas unidimensionales. Cada entrada de la tabla es una dirección base del segmento (dirección física inicial del lugar donde se aloja el segmentos en memoria RAM) y un límite del mismo (longitud del segmento). Finalmente, la traducción consiste en localizar el segmento y luego sumar al valor de la dirección base el byte que se quiere acceder. El SO es el encargado de controlar que la dirección física esté dentro del segmento.

COMPARACIÓN ENTRE LA PAGINACIÓN Y LA SEGMENTACIÓN

Característica	Paginación	Segmentación
¿El programador debe conocer la técnica?	NO	SI
Cantidad de espacios de direccionamiento por cada programa	UNO	MUCHOS
¿El tamaño del programa puede exceder la memoria física?	SI	SI
¿Se diferencian los procedimientos de los datos?	NO	SI
¿Se pueden proteger por separado los diferentes tipos de datos?	NO	SI
¿Se facilita la compartición de procedimientos entre usuarios?	NO	SI
Se genera fragmentación...	INTERNA	EXTERNA
Los programas se dividen en partes de tamaño...	FIJO	VARIABLE

- **Segmentación con paginación:** usada por los procesadores Pentium de Intel. En esta, los segmentos se dividen en páginas para no mantenerlos enteros en la memoria RAM. De forma combinada, permiten potenciar las ventajas de cada técnica.
- **Windows:** posee una implementación con paginación por demanda, no implementa la prepaginación, y cada un segundo un demonio de paginación verifica si existen suficientes marcos libres en RAM.
- **Linux:** implementa paginación con demanda sin prepaginación y sin el concepto de conjunto de trabajo. Usa el algoritmo de reemplazo de páginas del reloj.